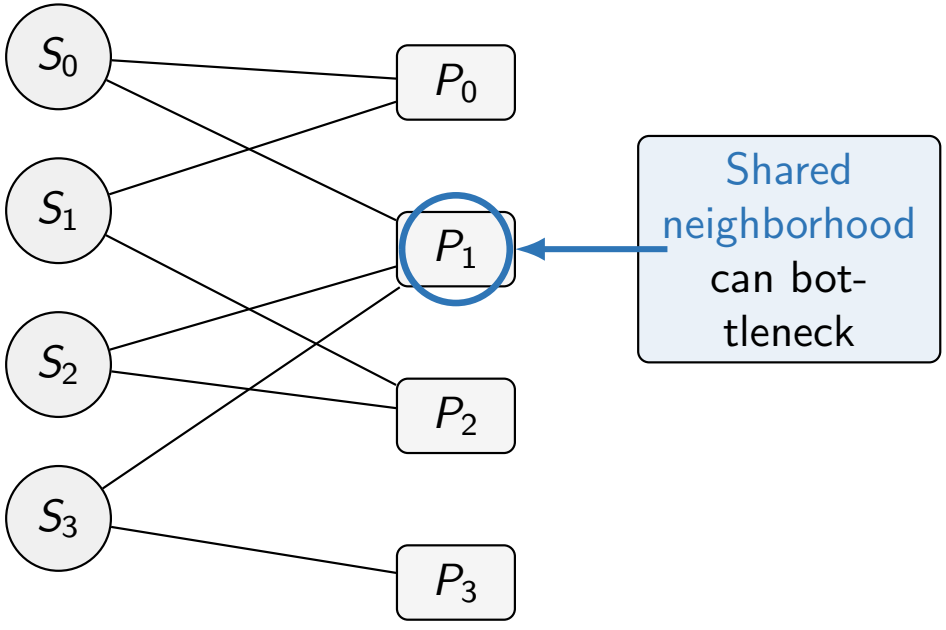
Optimising the RL Policy for Octopus CXL Memory Pooling

Pulak Mehrotra

March 2026

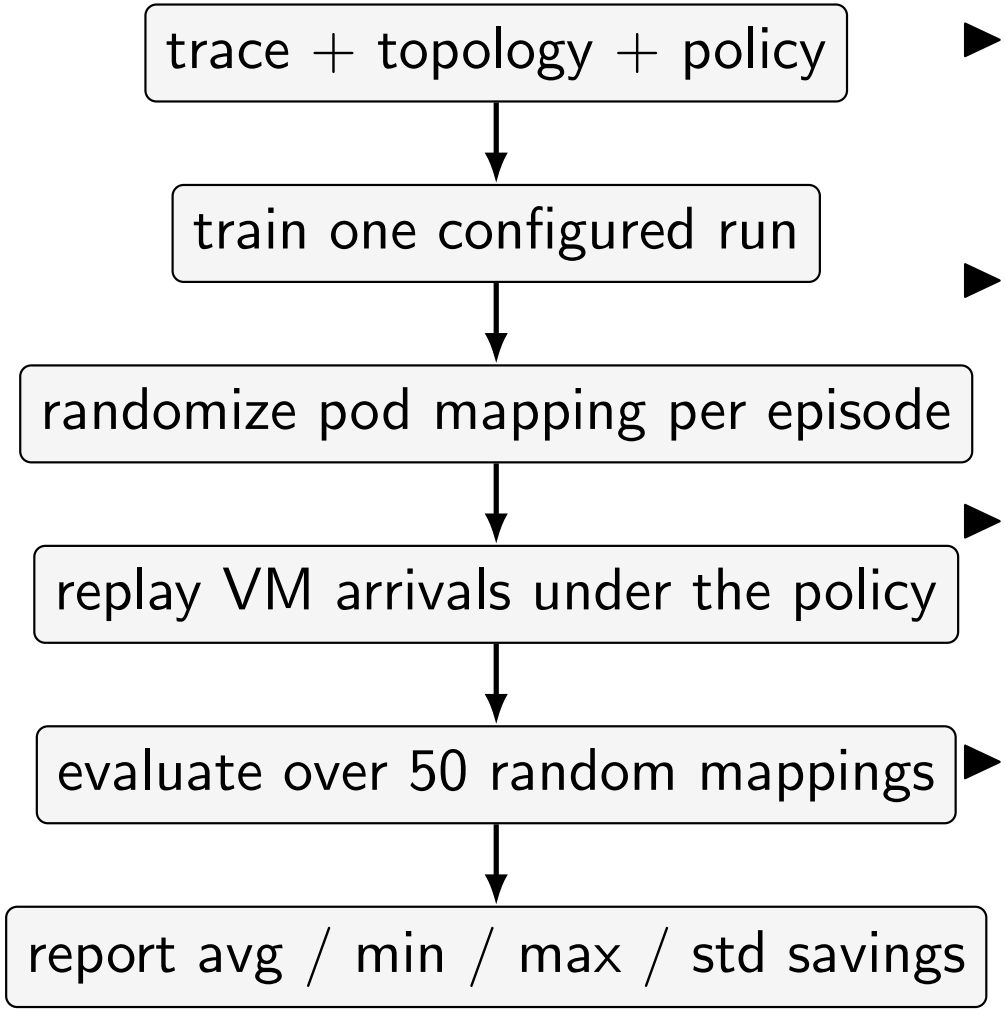
Recap: Octopus topology makes the allocation problem structurally hard.



- Each VM arrival can use only the host's reachable MPDs.
- Multiple servers can collide on the same small MPD neighborhood.
- There is no cheap migration loop to repair a bad early placement.
- The real objective is the worst MPD peak, not average load.

Provisioning is set by the hottest MPD.

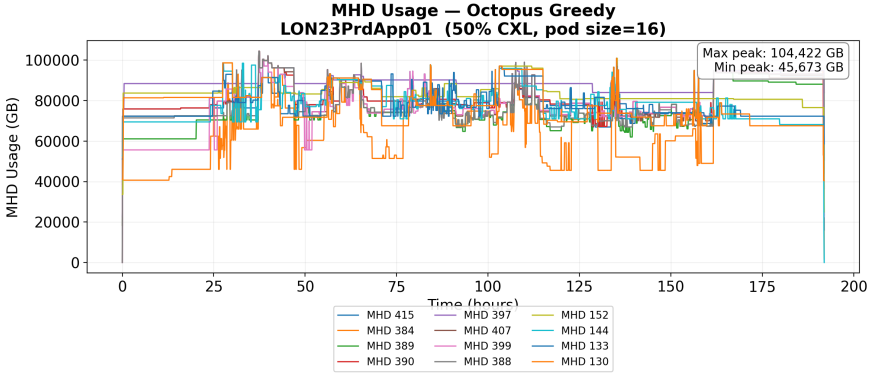
The current implementation already evaluates capacity outcomes across randomized pod mappings.



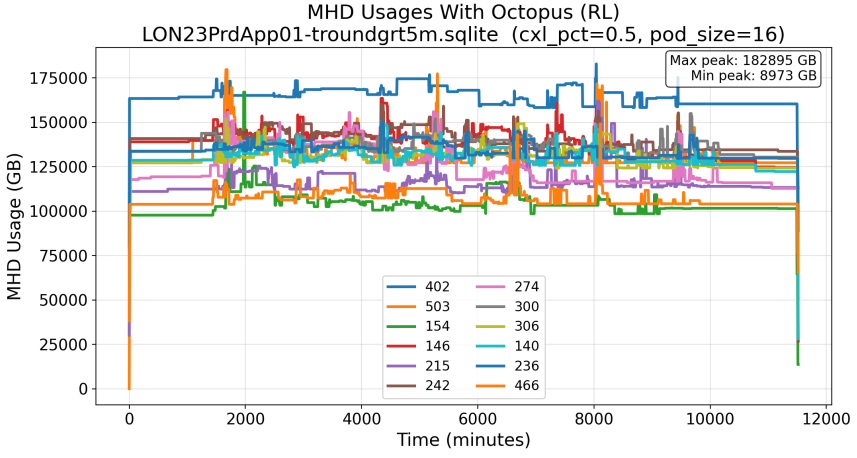
- The code path is not scoring a single cherry-picked topology.
- Training is still simple: one trace and one configured topology per run.
- Evaluation already reports a distribution over randomized mappings.
- The metric to trust is pooling savings or required per-MPD capacity, not raw RL reward.

The current SAC policy collapses load onto a few MPDs.

Preliminary



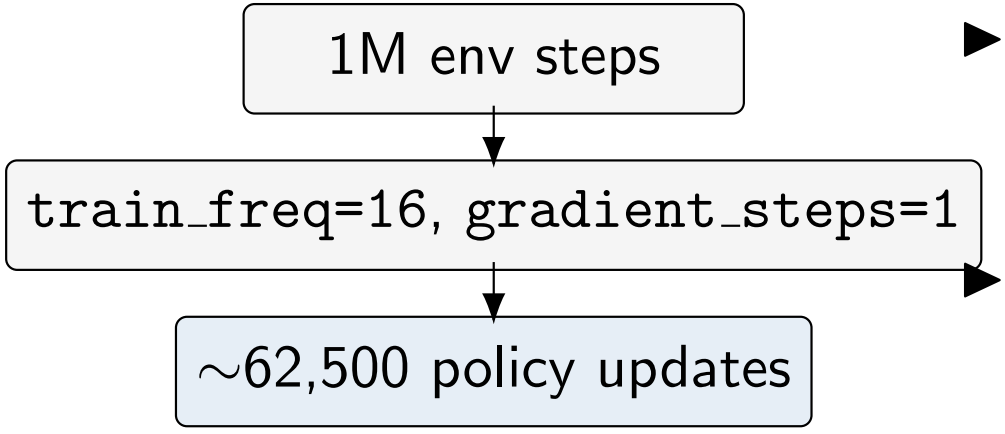
Greedy



Current SAC

- The clearest current failure signal is about a 20x max/min MPD imbalance.
- Behaviorally, the policy is not yet acting like a load balancer, which is a stronger diagnosis than “reward is noisy”.
- This is the evidence that motivates changing the task formulation before changing the deployment story.

1M samples with fast config yields ∼62k policy updates and no meaningful policy difference.



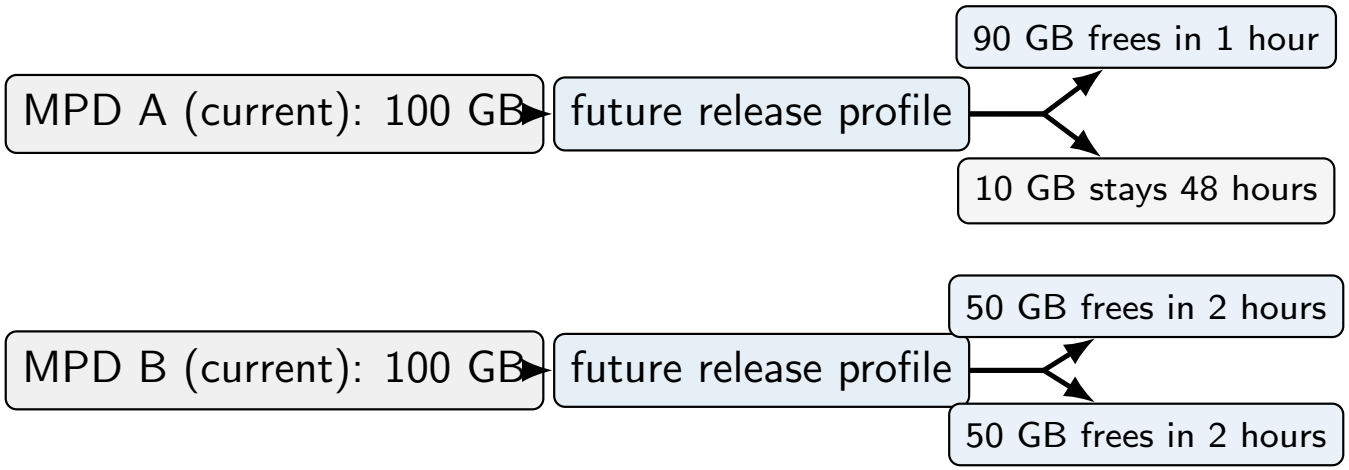
- Standard vs fast config: **no meaningful difference** in policy quality.
- Policy optimized fairly quickly—converged within 62k updates.
- Takeaway: the bottleneck is not training volume; it is state and reward design.

What I'm drilling down on: three fixes.

1. **State** — expose future pressure, not just current load
2. **Reward** — score balanced low load, not only the current peak
3. **Learning methodology** — learnability first, then algorithm choice

The next slides spell out each fix.

Fix 1: The state should expose future pressure instead of only current load.



- The current state mostly treats each MPD as a single instantaneous load number.
- Two MPDs can look identical now while having very different near-future headroom.
- Design principle: represent how much memory comes back and when it comes back.

The proposed state adds future-pressure features per reachable MPD.

Current state (unchanged)

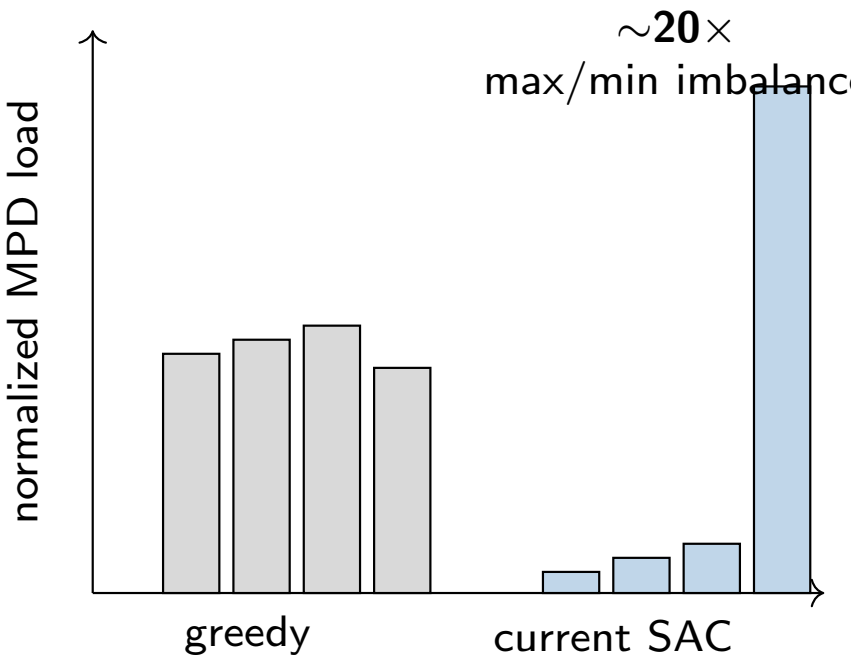
- Reachable MPD loads c_j
- Connectivity mask
- Incoming VM size
- Global peak load
- Hour-of-day encoding

Added or changed

- **Short-horizon release**: memory expected to free in next 1–2 hours
- **Medium-horizon release**: memory expected to free in 2–24 hours
- **Sticky load**: memory expected to stay beyond 24 hours
- **VM count** per reachable MPD

Temporal data: Diagram maps to buckets—short (1–2 hr), medium (2–24 hr), sticky (>24 hr). MPD A: 90→short, 10→sticky. MPD B: 100→medium. These features distinguish MPDs that look identical by current load alone.

Fix 2: The reward should score balanced low load instead of only the current peak.



- The reward is **sparse**: most signal comes from whichever MPD is currently worst.
- Bad choices among non-peak MPDs can look nearly identical—no gradient to balance load.
- Diagnostics point to reward design as a major contributor to load collapse.

Next: propose a denser reward.

Why the current reward is the problem.

Current immediate reward

$$r_t \propto -\Delta \max_j c_j(t)$$

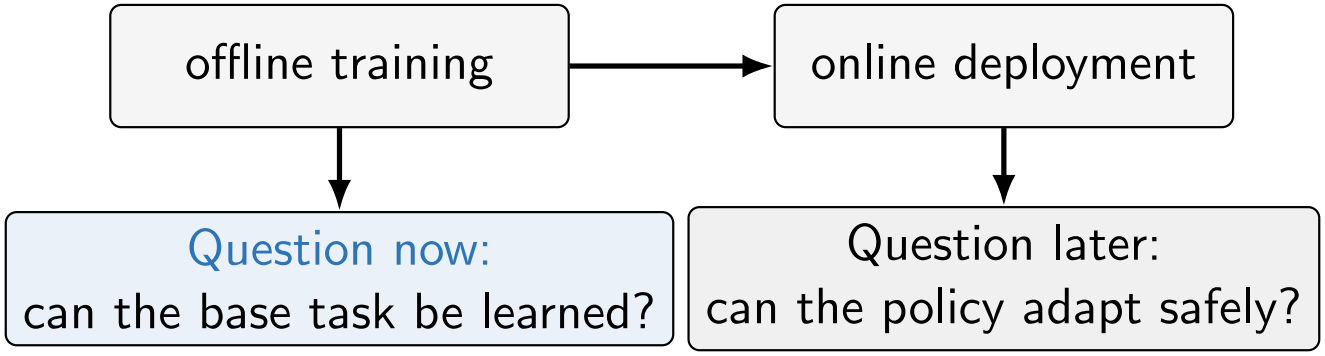
- Most of the signal comes from whichever MPD is currently worst.
- Bad choices among non-peak MPDs can look nearly identical.

Proposed immediate reward

$$R_t = \alpha\,\mathrm{Peak}(t) - \beta\,\mathrm{Var}(c(t)) - \gamma\,\mathrm{OverCap}(t)$$

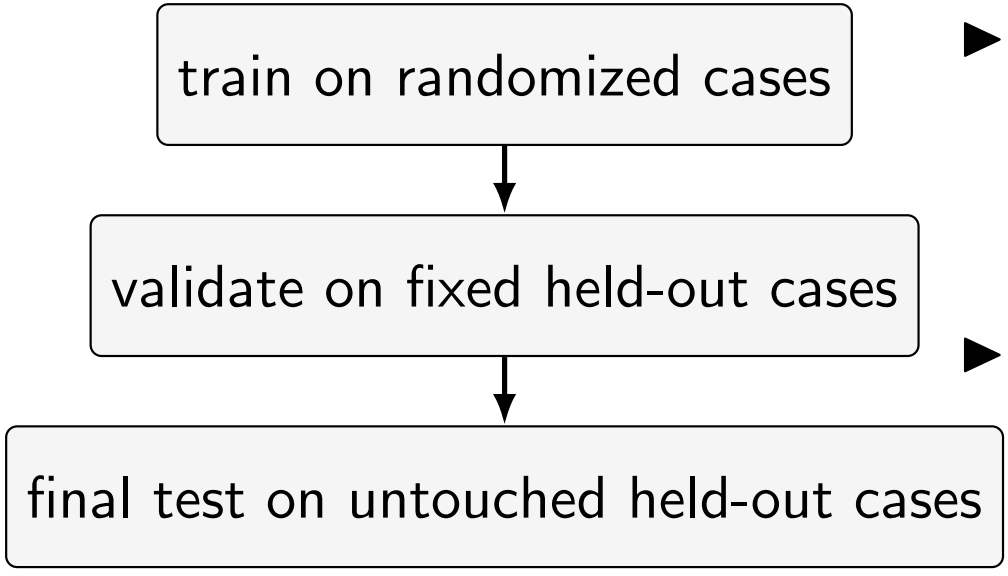
- The peak term keeps the reward aligned with required capacity.
- The variance term gives dense feedback across all MPDs.
- The over-cap term keeps capacity violations explicit.

Fix 3: The current bottleneck is learnability rather than catastrophic forgetting.



- The present failure appears during offline training, before any deployment drift.
- That makes state, reward, and training protocol the immediate issues.
- Same framing as robotics: learn the task in sim first; adaptation to real-world drift comes later.

The next round should separate learnability claims from generalization claims cleanly.



- Randomize traces, time windows, pod mappings, and CXL fractions during training.
- Use validation to select checkpoints, not final test traces.
- Keep the final test set untouched until the end.
- Run ablations in this order: state, reward, training, then algorithm.

First success criterion: beat greedy on held-out offline cases.

SAC: off-policy actor-critic with entropy regularization.

What it is

Soft Actor-Critic [1] maximizes expected return plus policy entropy. It uses a replay buffer and learns from off-policy data, making it sample-efficient for continuous actions.

Why it matters for Octopus: SAC is our current baseline. Its continuous action space fits the allocation-split problem. The replay buffer can mix data from different topologies and traces—which helps sample efficiency but may hurt generalization if the mix is wrong.

PPO: on-policy policy gradient with a clipped objective.

What it is

Proximal Policy Optimization [2] updates the policy using only on-policy data and clips the probability ratio to limit step size. No replay buffer.

Why it matters for Octopus: PPO is the clean control to test whether the base task is learnable. If PPO fails where SAC fails, the problem is likely state or reward design, not replay-buffer effects.

LCPO: online adaptation without catastrophic forgetting.

What it is

LCPO [3] adapts the policy online in non-stationary, context-driven environments. It constrains updates to preserve behavior on recent context while adapting to new regimes.

Why it matters for Octopus: LCPO addresses deployment-time drift (e.g., new datacenter, workload shift). It does not fix an unlearnable offline task—use it only after an offline policy already beats greedy.

Algorithm choice should follow evidence from SAC to PPO to LCPO.

Offline learnability	Online adaptation
▶ SAC stays as the current continuous-action baseline. ▶ PPO is the clean next contro because it removes replay-buffer mixing and tests whether an on-policy learner can solve the base task. ▶ If PPO still fails, the right conclusion is that task formulation is still wrong.	▶ LCPO is the deployment-time method for non-stationary contexts. ▶ Its value is forgetting resistance, not fixing an unlearnable offline task. ▶ It should enter only after offline policies already beat greedy consistently.

Decision rule: if PPO does not work after state, reward, and training fixes, do not jump to LCPO.

The next milestone is a credible offline result that beats greedy on held-out cases.

- Finish the state and reward redesign, then rerun offline training with a larger budget and cleaner train/validation/test separation.
- Add learning evidence: training/validation curves that show whether the policy is actually learning (not just a final checkpoint).
- Report four quantities together: pooling savings, MPD imbalance, oversubscribed MPDs, and variance across randomized pod mappings.
- Use PPO as the immediate control once the measurement path is clean enough to compare against SAC.
- Defer LCPO and other online machinery until there is one offline result that looks trustworthy to a systems audience.

Advisor feedback on the experimental scope and decision rule would be most useful now.

Questions for feedback

1. Is "beat greedy on held-out offline cases" the right near-term bar before any online adaptation work?
2. Is the proposed order of ablations correct: 1. state, 2. reward, 3. training protocol and algorithm?
3. Can we get more data here?

Stakes: (1) Success criterion for next phase. (2) Order isolates causes. (3) 10 traces may suffice for train/val/test; need to know before scaling up.

References I

[1] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, "Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor," in *Proceedings of the 35th International Conference on Machine Learning (ICML)*, 2018.

[2] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," *arXiv preprint arXiv:1707.06347*, 2017.

[3] P. Hamadanian, A. Nasr-Esfahany, M. Schwarzkopf, S. Sen, and M. Alizadeh, "Online reinforcement learning in non-stationary context-driven environments," in *International Conference on Learning Representations (ICLR)*, 2025. Spotlight.

Milestones dashboard (what I will bring next).

Milestone	What I will do	What I will show
Fix 1: state	Add future-pressure features per reachable MPD (short / medium / sticky + VM count)	State-ablation table: load-only vs +pressure; savings + imbalance + over-cap
Fix 2: reward	Replace peak-only proxy with denser immediate reward (peak + variance + explicit over-cap)	Reward-ablation table + one failure-analysis plot (per-MPD load timeline)
Fix 3: learning methodology	Clean train/val/test separation; larger offline budget; disciplined reporting	Learning evidence: train/val curves + held-out savings (not just a final checkpoint)
Systems reporting	Report metrics together across randomized pod mappings	One result table: pooling savings, MPD imbalance, oversubscribed MPDs, variance
Scope gate	Defer online adaptation machinery until one offline result is credible	Clear stop/go: offline beats greedy on held-out cases before LCPO